USC University of Southern California

ECE 105: Introduction to Electrical Engineering

Lab 4.2

Camera classifier

Yasser Khan

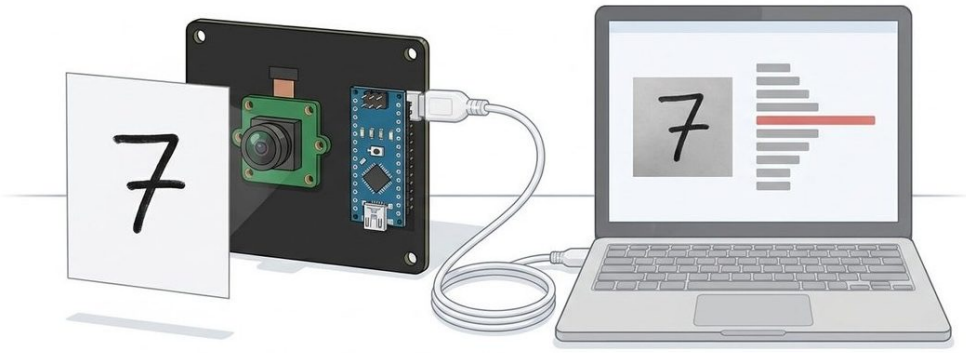
Rehan Kapadia

Today's mission

Show the board's camera a digit you wrote, and let your Lab 4.1 network read it.

By the end of this lab you can:

- Stream **live pictures** from the camera into the notebook
- Turn a picture into a **28 x 28 MNIST-style input**
- Classify your digits live with your **Lab 4.1 CNN**
- Measure the **domain shift** and close part of the gap

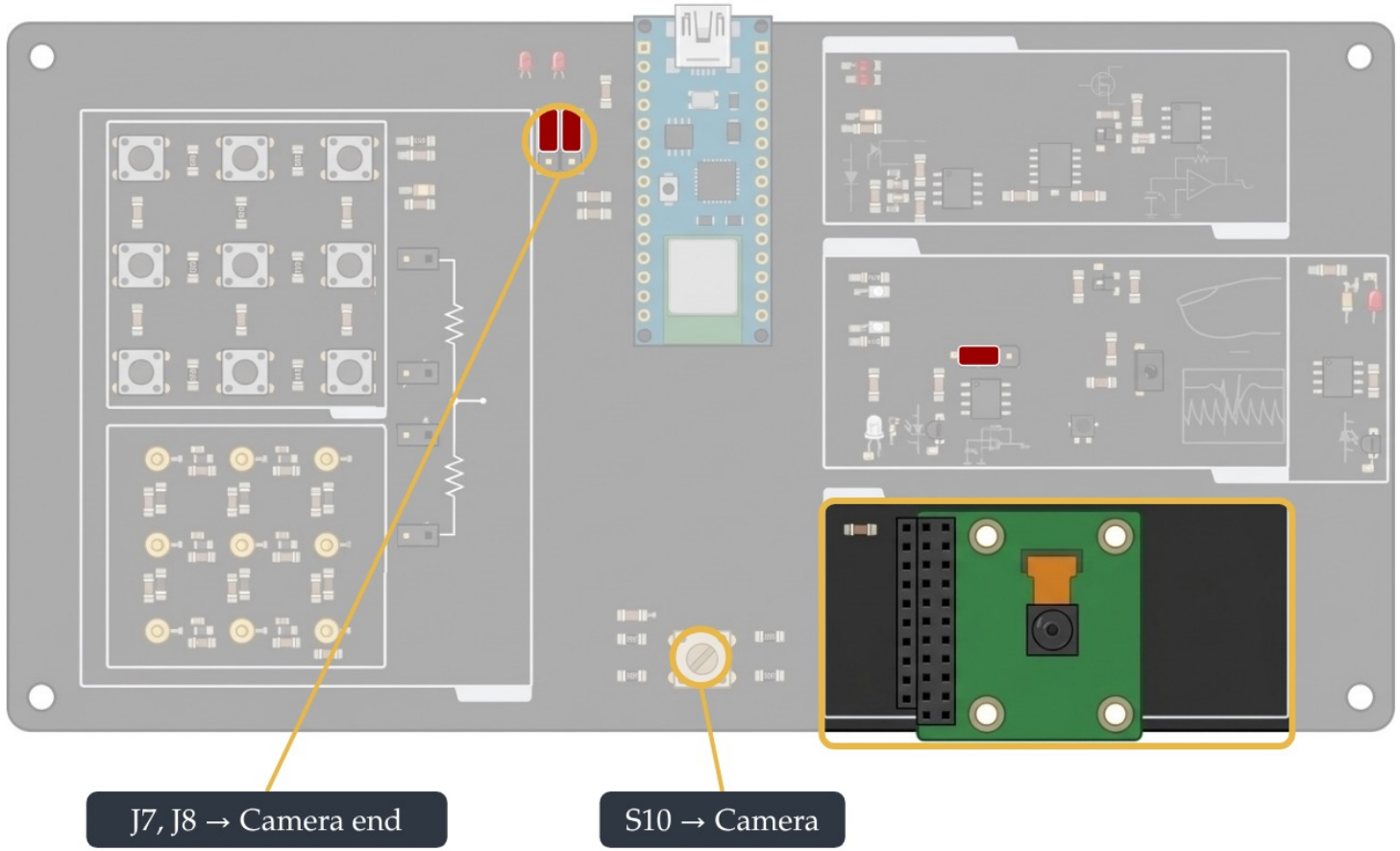


A paper digit, the board's camera, and the network's answer.

A	B	C	D	E	F	G
Camera **10 min**	Live view **15 min**	Preprocess **10 min**	Load CNN **5 min**	Predict **15 min**	Test set **25 min**	Domain shift **30 min**

SETUP

The camera lab moves the jumper caps. Move them back to Module 1 when you are done.



1. **S10 → Camera**
   Turn the selector to Camera.

2. **J7, J8 → Camera end**
   Both caps on pins 1-2: A0 and A1 go to the camera.

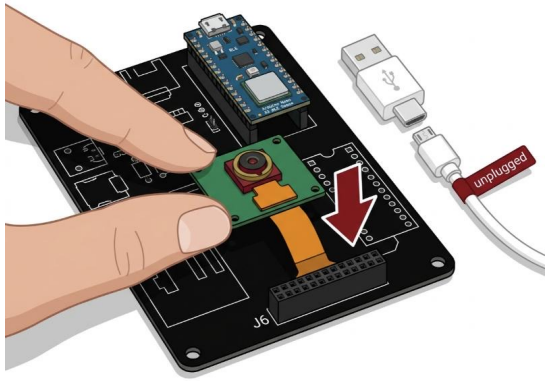
3. **Camera on J6**
   The OV7675, lens facing out. Seat it with the USB cable unplugged.

4. **Digits on paper**
   0 to 9 in thick black marker, one per 4 cm square.

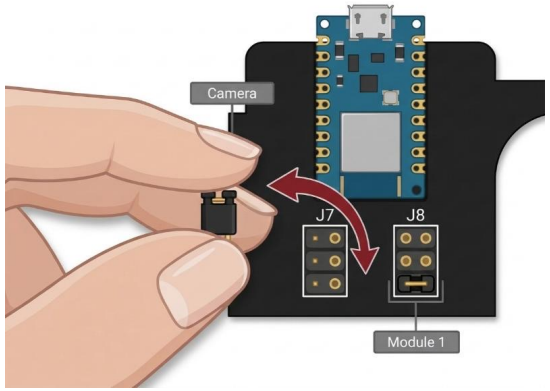
Part A: Connect the camera

Unplug, seat the camera, move the caps, plug back in, run the first cell.

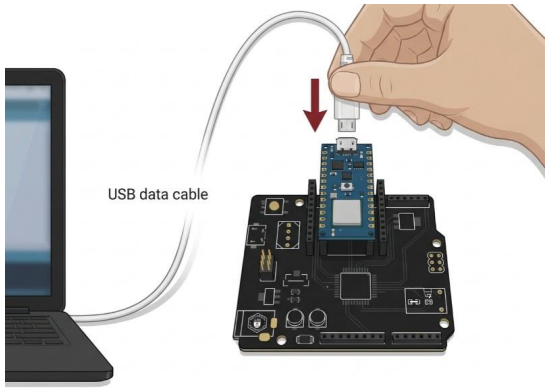
PART A **Goal: A1 · DO** connect the camera and upload its firmware.



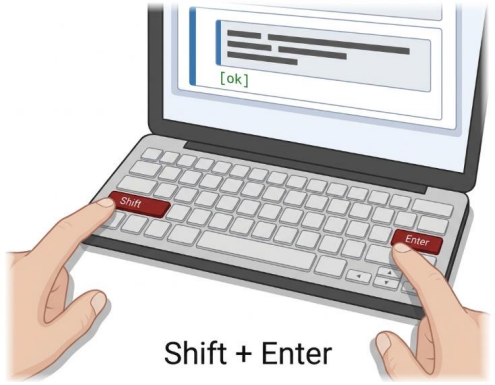
1. USB unplugged: seat the camera on J6



2. J7, J8 (3-pin headers): caps to pins 1-2, the Camera end



3. Plug the USB cable back in



4. Run the first cell with Shift+Enter

You should see: two **[ok]** lines. The first time, **connect_camera** compiles and uploads the camera firmware: about a minute.

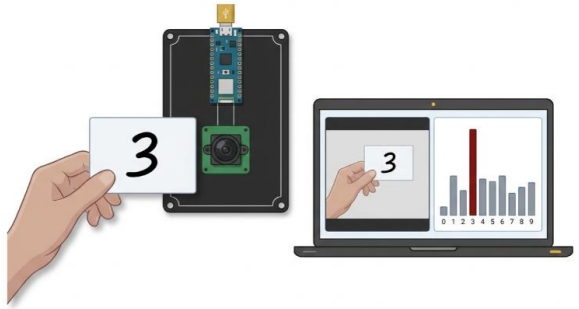
Part B: Look through the camera

The Nano reads a frame, scales it to 160 x 120 and sends it over USB as base64 text.

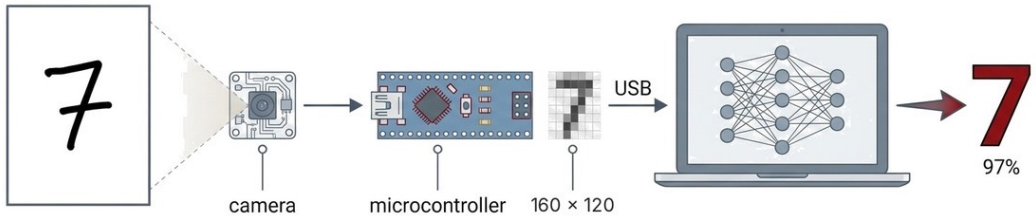
PART B

Goal: look through the camera, fix the orientation, time the pictures.

- **B1 · DO cam.live(15)**: a digit **in front of the lens**, 10 to 20 cm away
- **B2 · ANSWER** Upside down or mirrored? **FLIP_UD**, **FLIP_LR**
- **B3, B4 · RUN** One picture; time five pictures at two sizes
- **B5 · EDIT** Base64 characters, and their time at 115 200 baud



Digit facing the lens, filling about half the picture.



Camera, Nano, USB, laptop: the network runs on the laptop.

You should see: paper light gray, not black or burned out. A 160 x 120 picture is **25,600** characters: **2.2 s** at 115 200 baud, yet it takes only a few hundred ms.

Parts C and D: Network input and your CNN

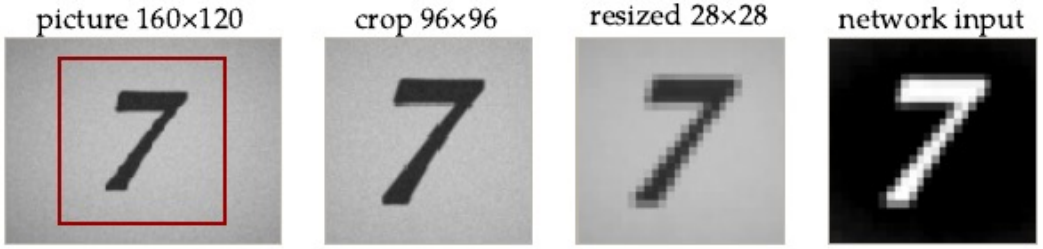
USC University of Southern California

to_digit: center crop, 28 x 28, invert, stretch; then the Lab 4.1 normalization.

PARTS C, D

Goal: turn a camera picture into a network input and load your CNN.

- **C1 · DO** Hold up a digit: crop, resize, invert, stretch
- Then (x - 0.5)/0.5, exactly as in Lab 4.1
- **D1 · RUN** Load **../data/models/mnist_cnn.pt**; if it is missing, D1 trains one (15 s)



160 x 120 picture, center crop, 28 x 28, network input.

Must match Lab 4.1 training

- white digit on black, 28 x 28
- digit about 20 px tall, centered
- pixels scaled with (x - 0.5) / 0.5
- the same SmallCNN class

You should see: a **white digit on black** that fills the middle of the 28 x 28 input, then **[ok] loaded your Lab 4.1 CNN** and an MNIST accuracy near **98.5 %**.

Check (Checkpoint 1): the live view shows your digit, and the preprocessing ends in a centered white digit on black.

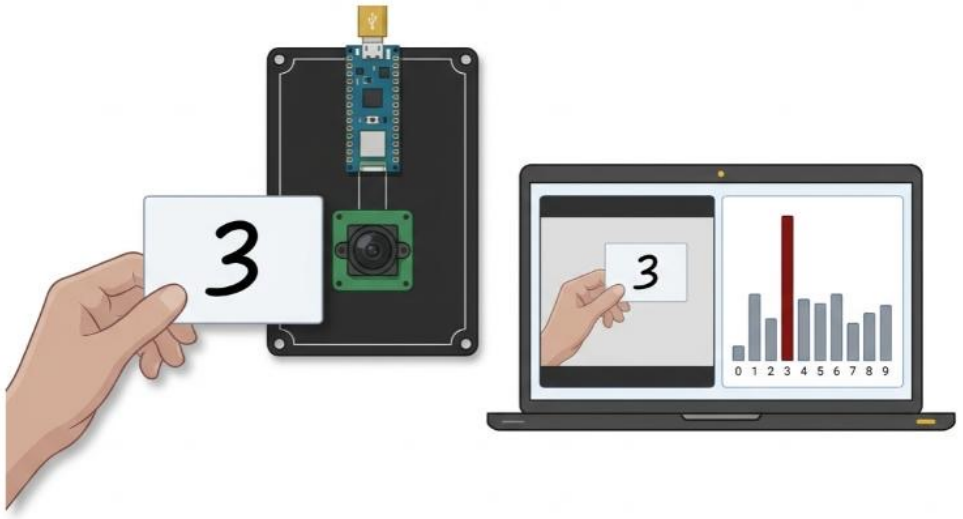
Part E: Live prediction

The largest of the ten softmax probabilities is the network's confidence.

PART E

Goal: predict your digits live and record three predictions.

- **E1 · DO** Live view with predictions: hold up different digits
- **E2 · DO** Three predictions: hold a digit, press Enter; three digits
- **E3 · EDIT** Softmax by hand from the ten logits z; compare with PyTorch



Hold the digit facing the lens; the tallest bar is the prediction.

You should see: a thick, clear digit at **95 % or more**; an ambiguous one (an open 3) near **40 to 50 %**.

Check (Checkpoint 2): the live view predicts your digits, and you have three prediction figures.

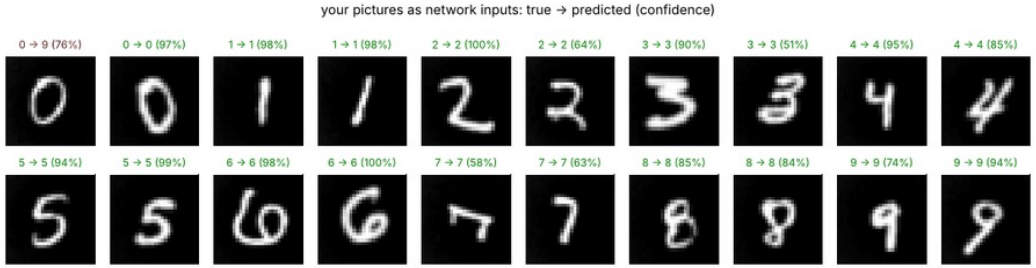
Part F: Your own test set

Two pictures of each digit make a 20-picture test set of your handwriting.

PART F

Goal: measure the accuracy and the confusion matrix on your handwriting.

- **F1 · DO** Run **collect**: for each digit, hold it up and press Enter
- Move the paper a little between the two pictures of a digit
- **F2 · RUN** The contact sheet of your 20 inputs
- **F3 · EDIT** Accuracy, and the confusion matrix **cm[true, predicted]**



Typical contact sheet: a red title marks a wrong prediction.

You should see: typically **60 to 90 %** on your 20 pictures, below the **98.5 %** on MNIST. Saved: my_digits.npz and my_digits_results.csv.

Check (Checkpoint 3): the contact sheet, your accuracy next to MNIST's, and the confusion matrix.

Part G: Domain shift and one improvement

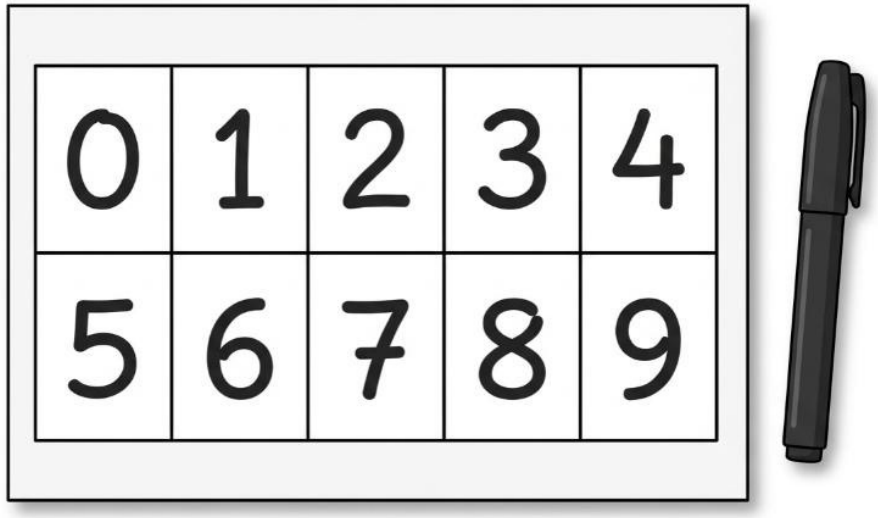
USC University of Southern California

MNIST and your pictures come from different domains. Measure the gap, then narrow it.

PART G

Goal: put numbers on the domain shift and try two improvements.

- **G1, G3 · RUN** Your inputs next to MNIST; once without the normalization
- **G2 · EDIT** Stroke fraction and digit height, yours against MNIST
- **G4 · EDIT** Crop sweep, 0.5 to 1.0 (G5 keeps the best crop)
- **G6 · RUN** Fine-tune on your first pictures, test on your second
- **G7, G8 · DO** One changed condition; a digit written on the spot



Marker and size matter: MNIST digits are about 20 px tall.

You should see: camera digits about **14 to 16 px** tall against **20** for MNIST; without normalization the accuracy falls sharply; best crop often **0.5 to 0.7**.

Check (Checkpoint 4): crop sweep, fine-tuning result, conditions table, and a live view that reads a digit you just wrote.

Stuck?

USC University of Southern California

Check these first. If the fix does not help, raise your hand.

Camera not found

Unplug USB, reseat the camera on J6 (lens out), plug in, press RESET.

The picture is black or burned out

Change the light or the distance: paper light gray, ink dark.

The digit is upside down or mirrored

Set FLIP_UD or FLIP_LR to True in B2 and rerun it.

[!!] mnist_cnn.pt not found

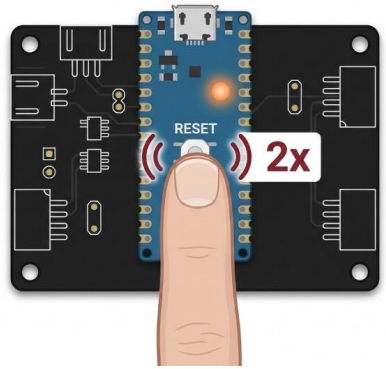
Finish Lab 4.1 (J3), or let D1 train a CNN now (15 s).

Poor accuracy on your digits

Thick marker, digit centered and half the height, no glare; try the crop sweep.

A [!!] line about the selector

Only a warning. Check S10 on Camera and both caps on the Camera end.



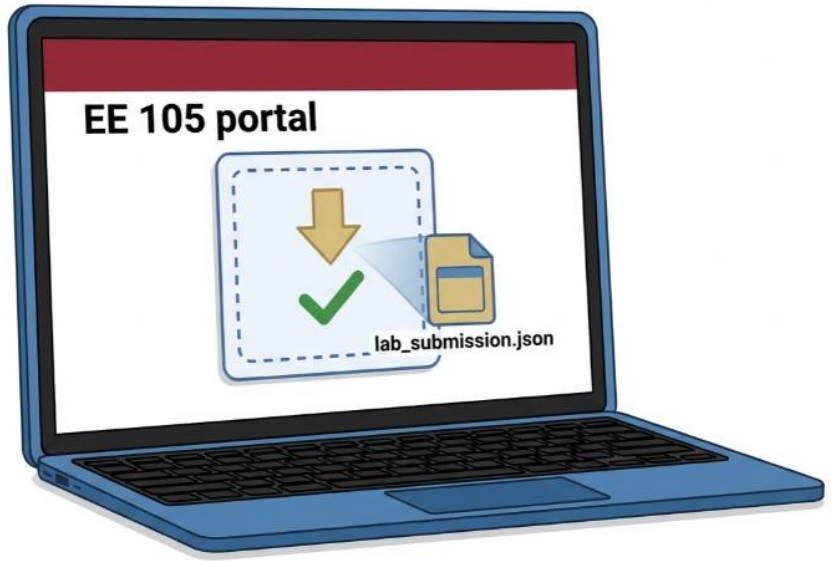
Upload stuck? Double-tap RESET, then rerun.

Submit your results

The last cell saves your results in one file. Upload it on the course portal.

```
lab4.2_submission.json
```

1. **Save · RUN** writes **lab4.2_submission.json** next to the notebook
2. It holds your numbers, the labels and predictions of your 20 pictures, and their 28 x 28 inputs
3. Log in at **ee105.org/portal** with your student ID and portal PIN; open Lab 4.2
4. Upload the file. It is graded automatically; you can upload again after a fix



ee105.org/portal

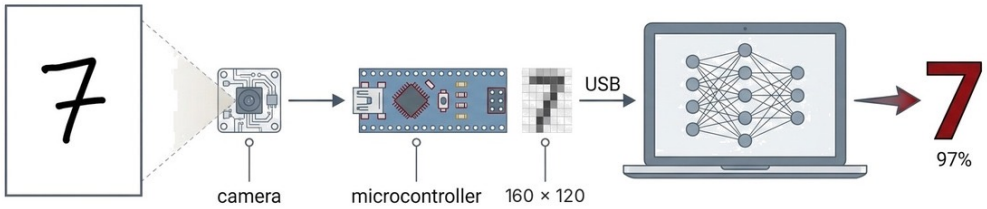
Check: before you leave, move both caps of J7 and J8 back to the **Module 1** end (pins 2-3).

Lab complete

You built a camera that reads handwriting with your own neural network.

You built:

- A live camera view inside the notebook
- One preprocessing for photos and camera pictures
- A 20-picture test set and its confusion matrix
- A crop and fine-tuning study of the domain shift



Camera, microcontroller, laptop: 97 % on a 7.

Did you know?

Running machine learning on a microcontroller is called TinyML. The Nano has 256 KB of RAM; your CNN needs 808 kB in float32, so it would first be cut to 8-bit numbers (Chapter 4).

Next

Lab 5.1, LiFi: send text from one board to another with light. Caps of J7 and J8 back on the Module 1 end now.